Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

After reading *Attention Is All You Need*, this felt like the natural next paper to study.

The Transformer paper helped me understand how modern language models process context inside a sequence. RAG moves the conversation one step further. It asks a very practical question:

What if the model does not have enough knowledge inside its parameters?

That question is extremely relevant to real-world AI systems. In production, we rarely want a model to answer only from what it memorized during training. We want it to use fresh, domain-specific, and verifiable information.

That is where Retrieval-Augmented Generation comes in.

RAG combines two things:

1. A retriever that finds relevant documents from an external knowledge base
2. A generator that uses those documents to produce an answer

In simple words:

The model does not just "remember."
It retrieves first, then generates.

The problem RAG is trying to solve

Large language models store knowledge inside their parameters. This is called parametric memory.

For example, when a model like GPT or BART answers a question, some of that knowledge comes from patterns stored in its weights during training.

That is powerful, but it has limits.

The model may not know recent information.
It may hallucinate.
It may struggle with specific domain knowledge.
It cannot easily update what it knows without retraining or fine-tuning.

RAG introduces non-parametric memory.

Non-parametric memory means external knowledge stored outside the model, usually in a collection of documents. Instead of forcing the model to memorize everything, we allow it to retrieve relevant information when needed.

This is the main idea:

- Parametric memory is what the model learned during training.

- Non-parametric memory is what the model can retrieve from an external knowledge base.

RAG combines both. That combination is what makes the paper important.

The high-level architecture

The RAG system has three main parts:

1. A query encoder
2. A retriever
3. A generator

The input question is first passed into a query encoder. The query encoder converts the question into a vector representation.

Then the retriever searches a document index and finds the top documents that are most relevant to the query.

These retrieved documents are then passed into the generator. The generator uses both the original question and the retrieved documents to produce an answer.

So the pipeline looks like this:

Question
→ Query Encoder
→ Retriever
→ Top-K Documents
→ Generator
→ Answer

This sounds simple, but the important part is that the retriever and generator are trained together end-to-end.

That means the system learns not only how to generate answers, but also how to retrieve useful evidence for those answers.

The three properties the authors wanted

From my understanding, the paper is trying to build a system with three useful properties.

First, retrieval and generation should work together. The system should not retrieve documents separately and then generate blindly. It should learn how retrieval supports generation.

Second, the retriever should work without needing labeled retrieval documents. In many real-world tasks, we have input-output examples, like question and answer pairs, but we do not always have labels telling us exactly which document should be retrieved. RAG learns retrieval in a latent way.

Third, the system should work across different sequence-to-sequence tasks. It should not be useful only for one specific task. It should support knowledge-intensive NLP tasks where external information matters.

That is why the paper focuses on knowledge-intensive tasks.

What RAG needs to work

To build a RAG system, we need a few components.

First, we need a pretrained generator model. In the paper, the generator is based on BART. BART is a sequence-to-sequence model. It takes an input sequence and generates an output sequence.

Second, we need a pretrained retriever. The paper uses Dense Passage Retrieval, also called DPR. DPR is based on a bi-encoder architecture.

Third, we need an indexed knowledge base. The paper uses large document collections such as Wikipedia. Each document is represented as a dense vector and stored in an index. When a query comes in, the retriever searches this index to find the most relevant documents.

So RAG combines:

- A generator
- A retriever
- A document index

This is very close to how many modern AI products work today.

Parametric and non-parametric memory

This is one of the most important ideas in the paper. A normal language model depends heavily on parametric memory. The knowledge is stored inside the model parameters.

If the model knows that Paris is the capital of France, that knowledge is encoded somewhere in its weights. But this is not always enough.

Suppose we are building an enterprise assistant. The assistant needs to answer questions about internal documents, recent tickets, policy changes, product specs, or system incidents. That information may not exist in the model's training data.

Even if it did, the model would not know the latest version.

RAG solves this by adding non-parametric memory. The knowledge is stored in external documents, not only in the model weights. This makes the system more flexible. If the knowledge changes, we can update the document index instead of retraining the entire model. That is a very practical advantage.

Dense Passage Retrieval

The retriever used in the paper is Dense Passage Retrieval. The idea is to encode both the query and the documents into vector space.

- The query encoder creates a vector for the input question.
- The document encoder creates vectors for all documents.
- Then the retriever compares the query vector with document vectors and retrieves the most relevant ones.

- The retrieval score is based on similarity, often using dot product.

A simplified version looks like this: $p(z|x) \propto \exp(d(z)^T q(x))$

Here:

- x - is the input query.
- z - is a retrieved document.
- q(x) - is the query vector.
- d(z) - is the document vector.

The dot product: $d(z)^T q(x)$

measures how similar the document is to the query.

If the document vector and query vector point in a similar direction, the score is higher. That means the retriever thinks the document is more relevant.

This is where MIPS comes in.

MIPS and why it matters

MIPS stands for Maximum Inner Product Search. The retriever needs to search over a large document collection and find documents with the highest dot product score against the query vector.

In small examples, this is easy. But in real systems, the document collection may contain millions of passages. Searching through every document one by one would be too slow.

MIPS makes retrieval scalable by allowing efficient search over large vector indexes. This matters because RAG is not only a modeling idea. It is a systems idea. If retrieval is too slow, the whole system becomes unusable. A real RAG system needs retrieval to be fast, reliable, and scalable.

That is why vector search and indexing are such important parts of modern AI infrastructure.

How documents are used by the generator

After retrieval, the generator receives the input query and retrieved documents. The generator does not just generate from the question alone. It generates conditioned on both: x, the input query and z, the retrieved document.

So the generation probability becomes something like: p(y|x,z). Here: y is the output sequence.

The generator is saying: Given the question and this retrieved document, what answer should I produce?

But there is one more important part. RAG does not rely on only one document. It retrieves the top K documents and marginalizes over them. That means it considers multiple possible retrieved documents and combines their probabilities.

This is important because the retriever may not always pick the single perfect document. By considering multiple documents, the model has a better chance of using useful evidence.

Marginalization over documents

The paper treats retrieved documents as latent variables. That means the model does not know with certainty which document is the correct one. Instead, it retrieves several possible documents and reasons over them probabilistically.

The general idea is: $$p(y|x) = \sum_{z \in topK} p(z|x)p(y|x,z)$$

This means: The probability of generating the output y given input x is calculated by summing over retrieved documents.

For each document: p(z|x) tells us how likely the document is to be relevant.

p(y|x,z) tells us how likely the generator is to produce the answer using that document.Then we combine these.

In simple words, the system asks:

- Which documents are relevant?
- What answer would I generate from each document?
- How do I combine these possibilities?

This is one of the most important parts of the paper. It is not just retrieve and paste. It is probabilistic retrieval-augmented generation.

RAG-Sequence and RAG-Token

The paper proposes two versions of RAG.

- RAG-Sequence
- RAG-Token

Both use the same general idea, but they differ in how they use retrieved documents during generation.

RAG-Sequence

In RAG-Sequence, the model retrieves documents and uses the same document to generate the full output sequence. So if the system retrieves a document zzz, it generates the entire answer conditioned on that document. The probability is marginalized at the sequence level.

A simplified form is: $$p(y|x) = \sum_{z \in topK} p(z|x)p(y|x,z)$$

This is easier to understand. The model says, for each retrieved document, what is the probability of generating the full answer? Then it combines those probabilities across documents. This works well when one document contains enough information to answer the question.

RAG-Token

RAG-Token is more flexible. Instead of using the same document for the full answer, RAG-Token allows the model to consider different documents for different generated tokens.

That means each token can be influenced by different retrieved documents. This is useful when the answer may require information from multiple sources.

The probability is marginalized at each token step.

A simplified idea is: $p(y|x) = \prod_i \sum_{z \in topK} p(z|x) p(y_i|x, z, y_{<i})$

Here: y_i is the current token being generated.

$y_{<i}$ means the tokens generated before it.

So for every token, the model considers the retrieved documents and decides how they influence the next word. This is more powerful, but also more complex.

Why RAG-Token is harder to decode

One thing I found interesting is that RAG-Token is harder during decoding. In normal generation, we can use beam search to keep the best hypotheses while generating token by token. But with RAG-Token, each token may depend on a mixture of documents. This makes the search space more complicated.

The model is not just asking: What is the next best token?

It is also asking: Which document helps generate this token?

This increases complexity during decoding. RAG-Sequence is simpler because the document is selected at the sequence level. RAG-Token gives more flexibility but introduces more complexity.

This tradeoff is very common in machine learning systems. More flexibility often means more compute and harder implementation.

Training objective

The paper trains the model by minimizing the negative log likelihood of the target output. In simple terms, the model is trained to assign high probability to the correct output sequence.

The loss looks like: $-\log p(y|x)$

Here, the model is penalized when it assigns low probability to the correct answer.

Since documents are latent, the model learns to retrieve useful documents indirectly through the generation objective. This is an important idea. The system does not necessarily need direct supervision saying: This is the correct document.

Instead, if retrieving a document helps generate the correct answer, the model learns to increase the probability of retrieving that kind of document. That is powerful because labeled retrieval data is often expensive or unavailable.

What the paper helped me understand

Before reading this paper, I understood RAG mostly as a product pattern.

[1] User asks a question.
[2] System retrieves documents.
[3] LLM generates an answer.

That is the common modern explanation. But the paper made me understand that RAG is deeper than that.

It is not just “search plus generation.” It is a probabilistic framework where retrieved documents are treated as latent knowledge sources. The model learns how retrieved evidence contributes to generating the final answer. That helped me understand why early RAG was such an important step. It gave language models a way to use external memory.

Why RAG matters in real systems

This paper is highly relevant to current AI systems. Most production LLM applications cannot rely only on model parameters. In healthcare, finance, enterprise software, legal tech, education, and internal engineering tools, the model needs access to domain-specific knowledge.

That knowledge may be private.
It may change frequently.
It may live across many systems.
It may need traceability.

RAG gives us a way to connect language models to external knowledge.

For example, in a healthcare setting, a model should not guess based only on its training data. It should retrieve relevant guidelines, policies, patient-specific context, or operational documentation from approved sources.

In an engineering setting, a model should not guess why a service failed. It should retrieve incident reports, pull requests, runbooks, and architectural decisions.

That is why RAG became such a major building block in applied AI.

Where RAG struggles

RAG is powerful, but it is not perfect. The biggest weakness is retrieval quality. If the retriever fails to retrieve the right documents, the generator cannot produce a reliable answer. This is one of the most important lessons from the paper. The generator can only work with what it receives. Bad retrieval leads to bad generation. This means the bottleneck often moves from the language model to the retrieval system.

In real-world systems, we have to worry about:

- retrieval accuracy
- document chunking
- metadata filtering
- index freshness
- ranking quality
- latency
- evaluation
- access control

A simple RAG system may work in a demo, but production RAG requires a lot more engineering.

Another limitation: documents are not enough

RAG retrieves text documents. That works well for many tasks. But some real-world systems are not just collections of documents. They are networks of relationships.

For example:

[1] A pull request modifies a service.
[2] That service depends on another service.
[3] An incident happened after the change.
[4] An architectural decision explains the dependency.

If this information is only stored as text chunks, a RAG system may retrieve related text, but it may not understand the actual structure. This is where my thinking connects strongly to Engram.

RAG gives LLMs external memory.

But Engram is trying to make that memory structured.

Where RAG breaks in real systems

Most explanations of RAG stop at architecture diagrams. In practice, that's not where the difficulty is. The actual failure modes show up in the retrieval layer.

The biggest issue is that **retrieval quality defines everything downstream**. If the retriever pulls slightly irrelevant or noisy documents, the generator will still try to produce a confident answer. This creates outputs that look correct but are grounded in the wrong context.

Another problem is **semantic similarity is not the same as relevance**. Dense retrievers optimize for vector closeness, not causal correctness. Two documents can be semantically similar but answer completely different questions. This shows up a lot in domains like healthcare or finance where wording overlaps but intent differs.

Chunking introduces its own issues. When large documents are split into smaller pieces, **context gets fragmented**. Important relationships across sections are lost, and the retriever may surface incomplete information that the generator treats as sufficient.

There is also the issue of **staleness**. Unlike parametric memory, external knowledge bases need to be updated continuously. If indexing pipelines lag behind real-world changes, RAG systems can produce outdated answers even though the architecture itself is correct.

From a systems perspective, RAG is not just about combining a retriever and a generator. It is about:

- how documents are chunked
- how embeddings are generated
- how retrieval is evaluated
- how results are filtered before generation

In most production setups, these decisions matter more than the choice of model itself.

Connection to Engram

Engram is built around the idea that engineering context should not live only in raw documents. It should be represented as entities and relationships.

In a traditional RAG system, we might retrieve chunks from:

- pull requests
- incident reports
- architecture documents
- runbooks

Then the LLM tries to reason from those chunks. That can work sometimes.

But for system-level reasoning, I think we need something more explicit.

If I ask: What could break if this service changes?

I do not want the model to rely only on semantic similarity. I want it to reason over relationships:

- Service A depends on Service B.
- PR X modified Service A.
- Incident Y involved Service B.
- Decision Z explains why the dependency exists.

That is why Engram combines graph-based structure with retrieval and LLM reasoning.

RAG is extremely important because it shows how language models can use external knowledge. But Engram pushes this idea further by asking:

What if the external memory is not just documents, but a structured knowledge graph?

That is the direction I find most interesting.

My production takeaway

The biggest production lesson from RAG is this: LLMs are not enough by themselves.

A useful AI system needs memory, retrieval, grounding, and evaluation.

The model may generate the answer, but the quality of the system depends heavily on everything around it.

- The retriever matters.
- The index matters.
- The data pipeline matters.
- The document structure matters.
- The evaluation loop matters.

This is why I see RAG less as a model architecture and more as an AI system architecture.

The paper helped me understand why modern LLM systems are not just about larger models. They are about connecting models to the right context at the right time.

My takeaway

RAG changed the way I think about language models. The Transformer paper showed how models can learn context inside a sequence. The RAG paper shows how models can access knowledge outside themselves.

That is the jump.

Instead of expecting a model to memorize everything, RAG gives it a way to retrieve relevant knowledge when needed. This makes AI systems more practical, more flexible, and more useful in real-world settings.

But it also introduces a new challenge.

Once retrieval becomes part of generation, the system is only as strong as its retrieval layer.

That is the part I keep thinking about while building Engram.

RAG answers the question: How do we give language models access to external knowledge?

Engram builds on that question and asks: How do we make that external knowledge structured enough for reliable reasoning?

That is why this paper feels like a direct bridge from Transformers to the kind of applied ML systems I want to build.